EDXSO AI ENGINEER INTERN

Assignment 2 – Scholarship Intelligence Crawler

Complete Execution, Demonstration & Screenshot Guide

Purpose of this document. It follows the assignment requirements and the demo flow: Discovery → Crawling → Extraction → Verification → Confidence Scoring → Storage → Update → Change Detection → Stale/Expired Detection.

GitHub Repository: https://github.com/Sohail700/-Scholarship-Intelligence-Crawler

1. Assignment Requirements –

The assignment asks for a working Scholarship Intelligence Crawler rather than only a design. The system should discover scholarship opportunities for Indian students, crawl sources, extract structured information, verify authenticity, calculate an evidence-based confidence score, store records, update them on repeated runs, detect changes, and identify stale or expired scholarships.

Minimum output required

Requirement	Minimum
Real scholarship records	20+
Verified against primary/official sources	15+
Confidence ≥ 95%	10+
Different source types	3+
Change detection examples	2+
Expired/stale detection examples	2+

Required record information

- Scholarship name and provider
- Official/primary source URL
- Application URL, where available
- Amount / benefit
- Eligibility and academic requirements
- Course / education level
- Income, age, gender, category, domicile and institution criteria where applicable
- Opening and closing dates
- Documents required
- Selection process and renewal requirements
- Current status
- Confidence score, verification status, last verified date and source evidence

Critical authenticity rule

Aggregator sites, blogs and articles may help discovery, but they must not be treated as the authoritative source. A verified scholarship must retain its official source URL and evidence used for verification.

2. Ubuntu / WSL Installation and Project Setup

These commands are for Ubuntu or Ubuntu running through WSL on Windows.

2.1 Open Ubuntu and locate the project

If the project ZIP is in your Windows Downloads folder, Windows drives are mounted under /mnt in WSL.

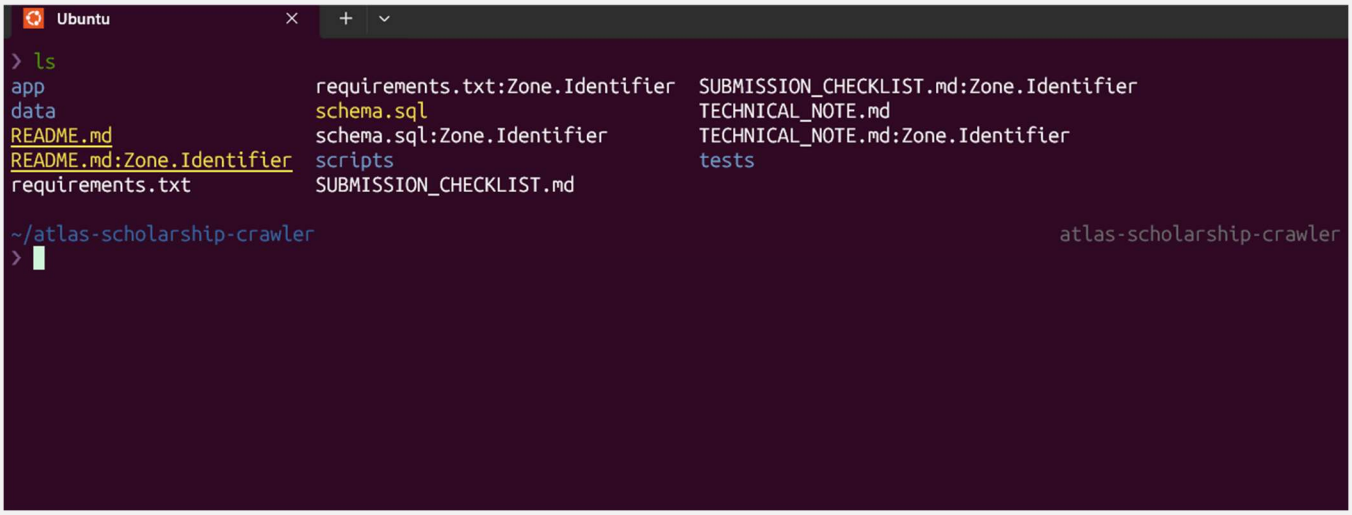
```
cd /mnt/c/Users/YOUR_WINDOWS_USERNAME/Downloads
ls
```

If unzip is not installed:

```
sudo apt update
sudo apt install -y unzip
unzip atlas-scholarship-crawler.zip
cd atlas-scholarship-crawler
ls
```

SCREENSHOT 1

Project folder opened in Ubuntu



Terminal showing the project directory and files such as app/, data/, scripts/, tests/, requirements.txt and README.md.

Scholarship crawler project opened in Ubuntu.

2.2 Install Python prerequisites

```
sudo apt update
sudo apt install -y python3 python3-pip python3-venv
```

Verify:

```
python3 --version
pip3 --version
```

2.3 Create and activate the virtual environment

```
python3 -m venv .venv
source .venv/bin/activate
```

After activation, your prompt should contain (.venv). Example:

```
(.venv) user@ubuntu:~/atlas-scholarship-crawler$
```

2.4 Install project dependencies

```
python -m pip install --upgrade pip
pip install -r requirements.txt
```

If installation completes without errors, the project dependencies are ready.

2.5 Configure environment variables

```
cp .env.example .env
```

The project can run without paid APIs. Do not add paid API credentials. The assignment requires free/open-source tools or genuinely free tiers.

3. Initialize and Verify the Database

3.1 Create the SQLite database

```
python -m app.cli init-db
```

This creates the SQLite database used by the crawler and dashboard. The database stores scholarship records, evidence, crawl runs, source snapshots and change events.

3.2 Load the included demonstration data

```
python -m app.cli seed-demo
```

The demonstration dataset allows the dashboard and assignment checks to be shown immediately. It also provides examples for verification, confidence scoring, change history and stale/expired status.

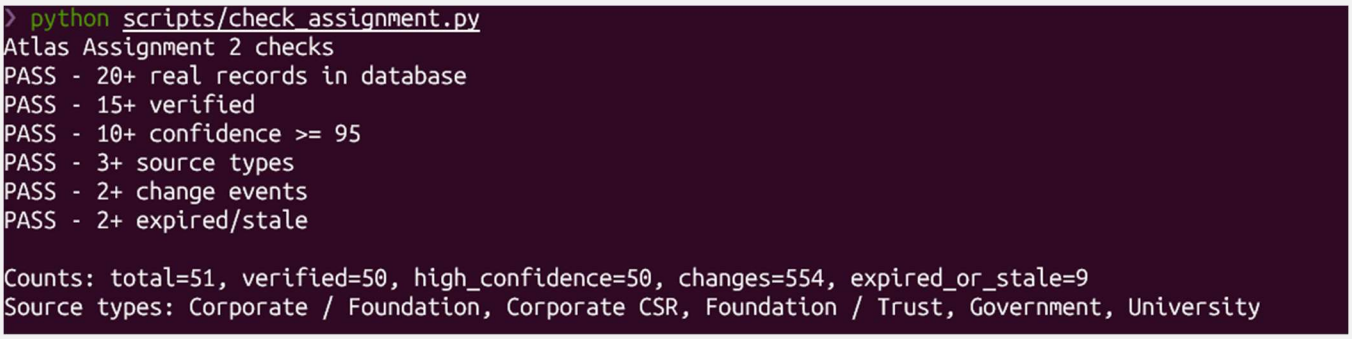
3.3 Run the assignment threshold checker

```
python scripts/check_assignment.py
```

You should see PASS messages for the minimum requirements. Keep this terminal output for your demonstration.

SCREENSHOT 2

Assignment threshold checker



Terminal showing the PASS results and the counts for total records, verified records, high-confidence records, change events and expired/stale records.

Automated check of assignment minimum requirements.

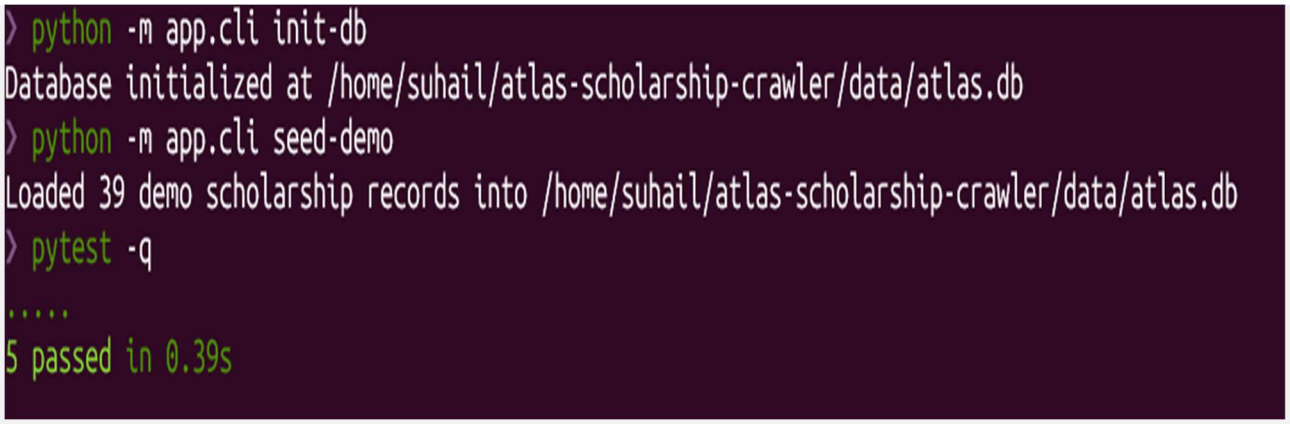
3.4 Run automated tests

```
pytest -q
```

The test suite checks core behaviours such as deterministic confidence scoring, change detection and discovery/link prioritisation.

SCREENSHOT 3

Test results



Terminal showing pytest completing successfully, ideally with the number of passed tests.

Automated project tests passing.

4. Understand the Project Before the Demo

The main pipeline is:

```
Discovery → Crawling → Extraction → Verification → Confidence Scoring → Storage → Update
```

The repeated-run behaviour adds:

```
Repeated Crawl → Compare Previous Snapshot → Detect Change → Record Event → Update Current Record
```

The project separates verification from availability. A scholarship can be verified as authentic while being expired or no longer available.

Main project components

Component	Purpose
app/discovery.py	Discovers relevant pages/links beyond a small seed list.
app/fetcher.py	Fetches pages and handles basic web retrieval.
app/extractor.py	Extracts scholarship information from unstructured source content.
app/verifier.py	Checks primary-source evidence and calculates deterministic confidence.
app/change_detector.py	Compares previous and current values and records changes.
app/database.py	Creates and updates the SQLite database.
app/dashboard.py	Streamlit dashboard for search and inspection.
app/cli.py	Command-line entry point for initialization, demo data and crawling.

Confidence score methodology

The confidence score must be based on actual evidence rather than an arbitrary number generated by an LLM. The implementation uses deterministic evidence checks. The score considers whether the source is official, whether the scholarship is present, whether an official application URL exists, whether eligibility and deadline evidence are present, currentness, field-level traceability and extraction consistency.

Important threshold
Score ≥ 95% → VERIFIED. Score below 95% → REVIEW_REQUIRED. Do not manually type a confidence number into the database.

Anti-hallucination approach

- Extracted values must be supported by source text.
- If a field is not stated by the source, store an explicit value such as “Not specified on the official source”.
- Important fields retain evidence so a reviewer can trace Database → Official Source → Evidence → Extracted Value.
- Aggregator information is not treated as authoritative evidence.

5. Start the Streamlit Dashboard

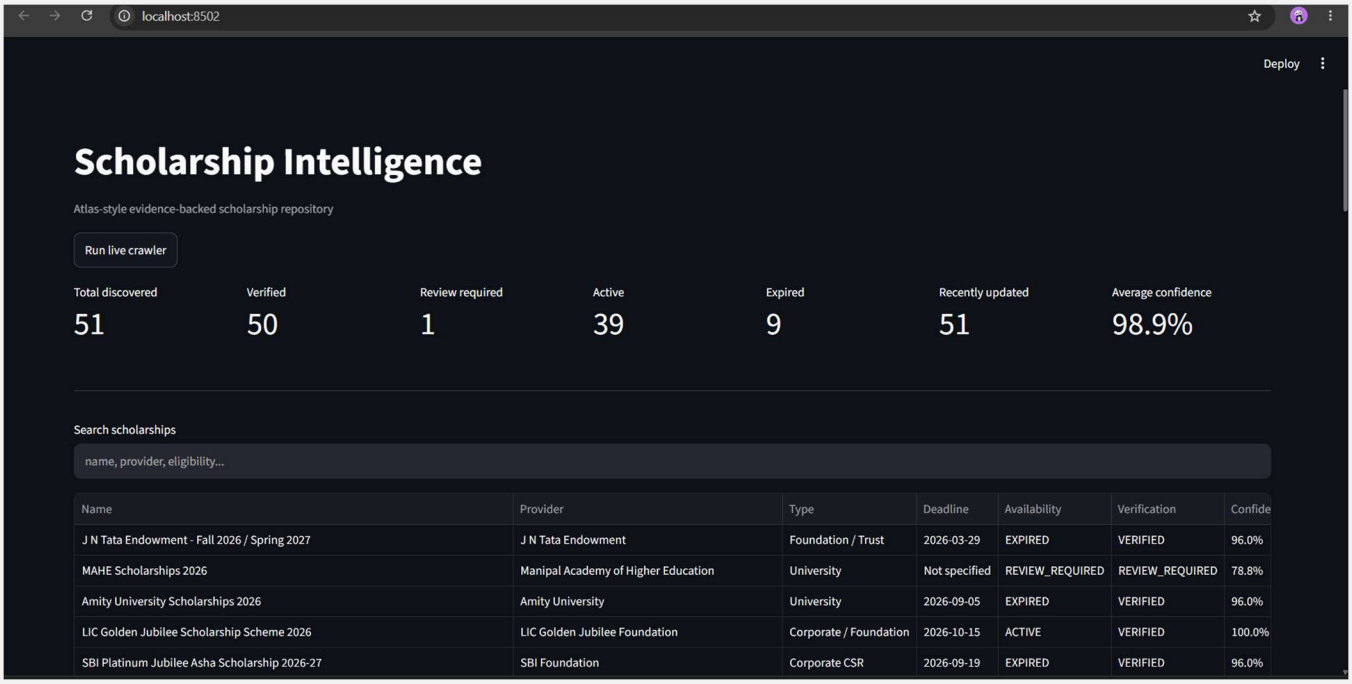
```
streamlit run app/dashboard.py
```

Open the displayed local URL, normally:

http://localhost:8501 0r http://localhost:8502

SCREENSHOT 4

Dashboard home page



Full Streamlit dashboard showing Scholarship Intelligence and the summary metrics.

Scholarship Intelligence dashboard.

5.1 Dashboard metrics to show

- Total discovered
- Verified
- Review required
- Active
- Expired
- Recently updated
- Average confidence

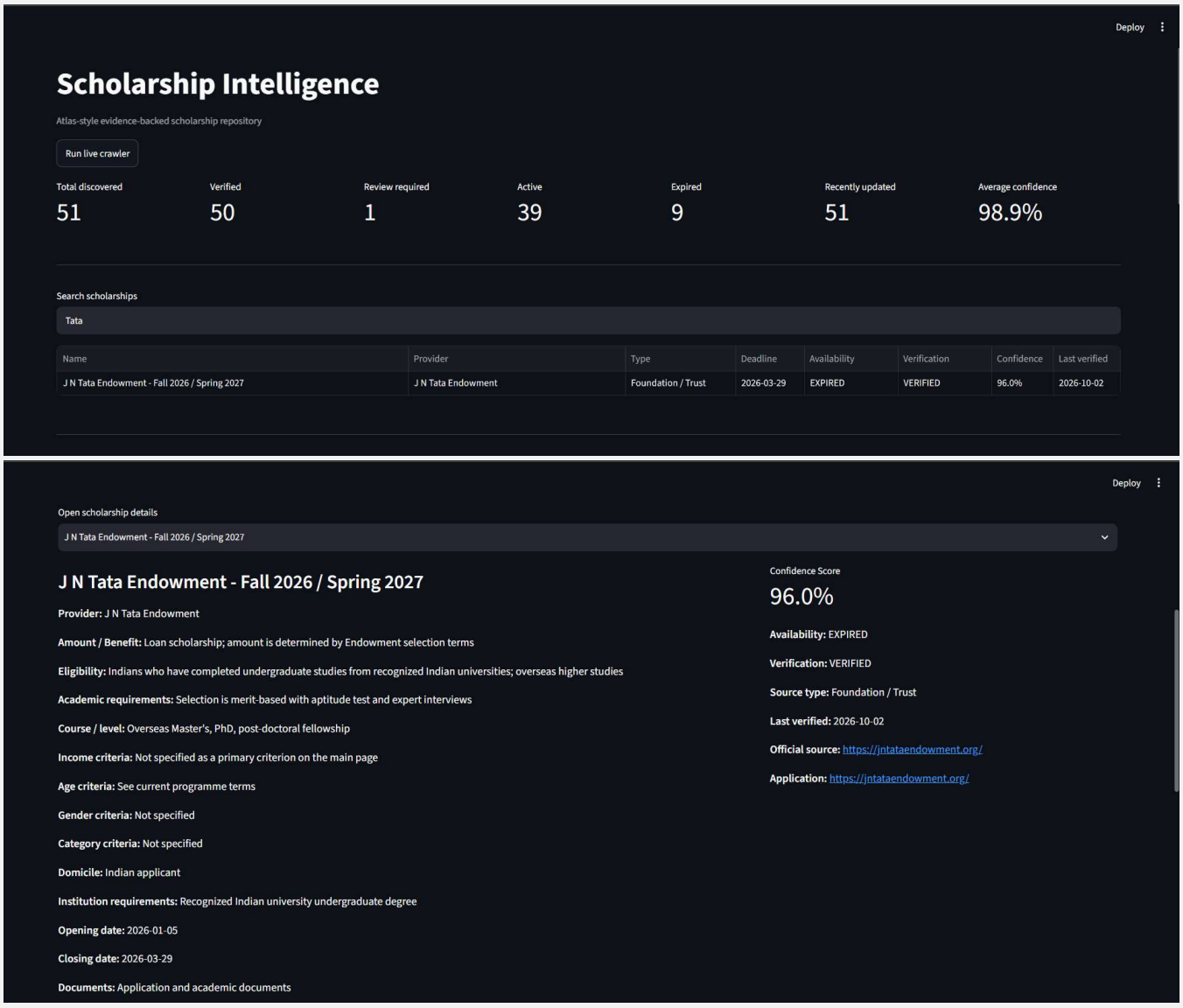
These metrics should be visible before opening an individual scholarship.

5.2 Search for a scholarship

Use the dashboard search/filter controls to find one of the demonstration records. Select a scholarship that has an official source, application link, confidence score and evidence.

SCREENSHOT 5

Searchable scholarship list



Dashboard list/table after searching for a scholarship. Make sure the scholarship name, provider, status and confidence are visible.

Searchable scholarship records.

5.3 Open scholarship details

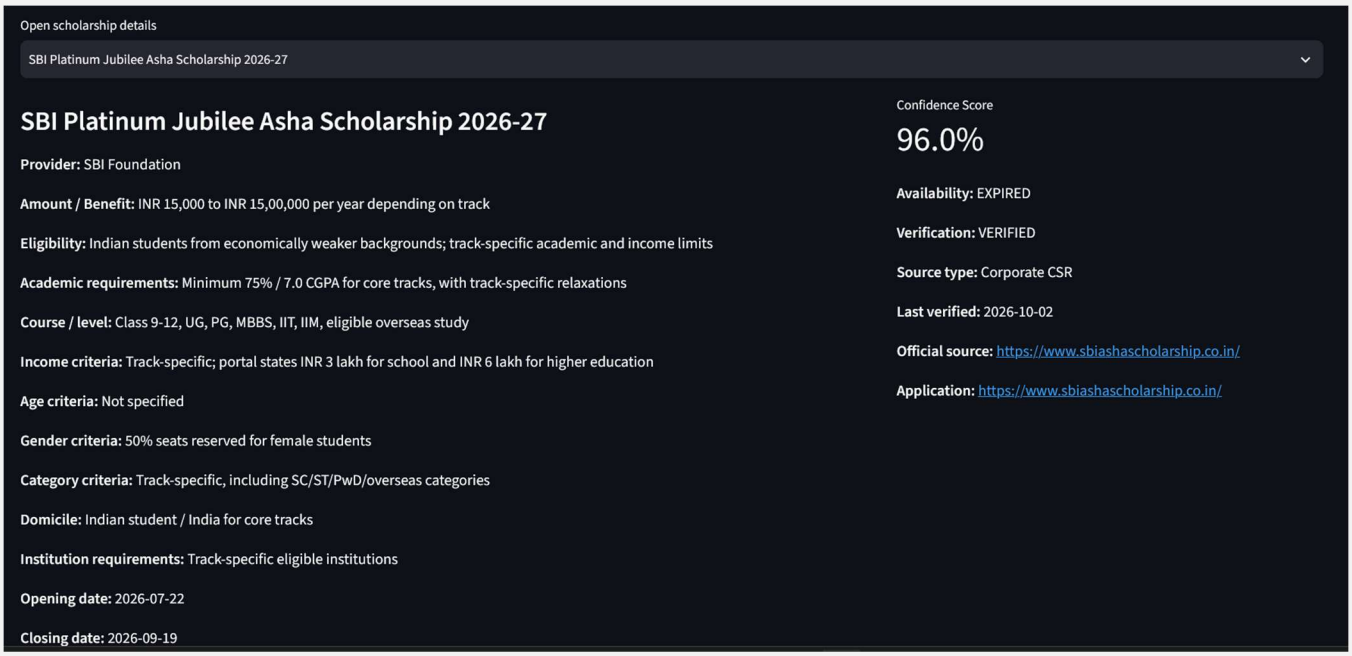
Open one record and show:

- Name
- Provider

- Amount / benefit
- Eligibility
- Deadline
- Official source
- Application link
- Status
- Confidence score
- Last verified
- Source evidence
- Change history

SCREENSHOT 6

Scholarship details



Individual scholarship details page/panel showing the structured fields and source URLs.

Detailed scholarship record.

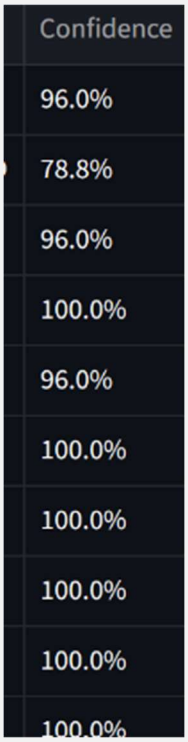
6. Demonstrate Confidence Scoring and Evidence

6.1 Show the confidence score

Open the confidence section for the selected scholarship. The important point is not just the percentage—it is that the percentage is produced from explicit evidence checks.

SCREENSHOT 8

Confidence score



Scholarship details showing the confidence percentage and verification status.

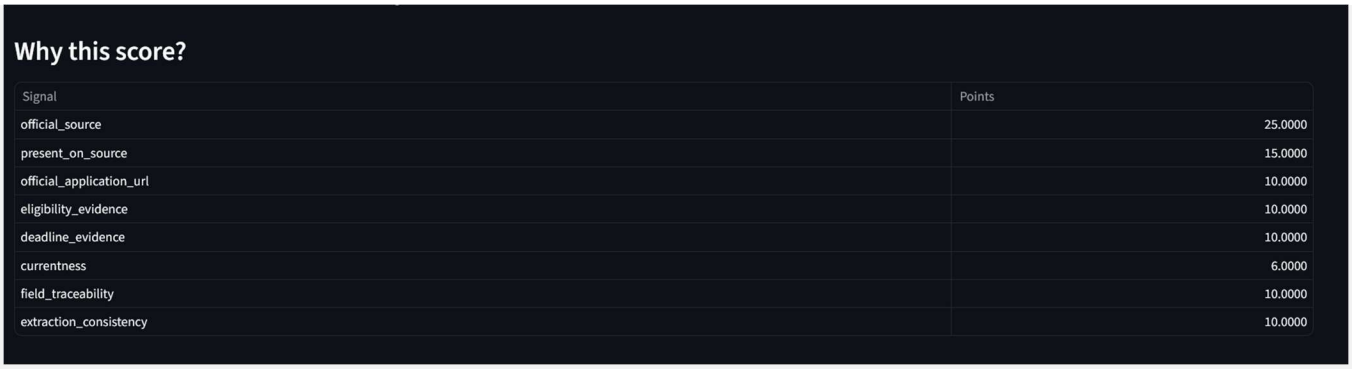
Evidence-based confidence score.

6.2 Show "Why this score?"

If the dashboard exposes the scoring explanation, expand it. Capture the individual checks or factors used by the system.

SCREENSHOT 9

Why this score?



Expanded confidence explanation showing the evidence factors/checks contributing to the score.

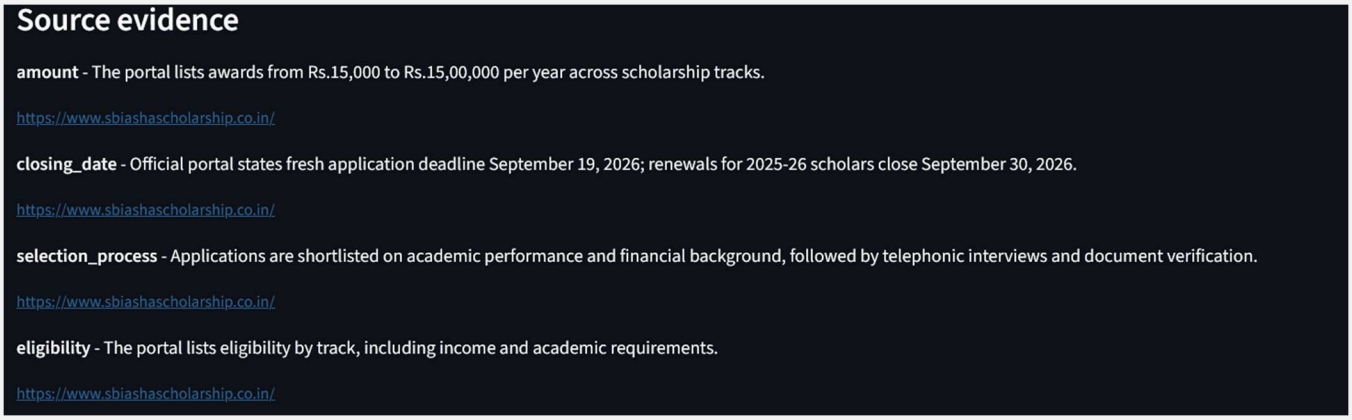
Confidence score explanation.

6.3 Show source evidence

Show the evidence attached to important fields. The reviewer should be able to see how a deadline, eligibility condition or other field was supported by the official source.

SCREENSHOT 10

Source evidence



What to capture: Evidence panel showing the official URL/source and the extracted supporting evidence for important fields.

Traceable source evidence.

7. Demonstrate Change Detection

7.1 Run the change demonstration

```
python scripts/demo_change.py
```

This demonstration creates/records example changes such as a previous deadline and a newly observed deadline. The system should retain the old value rather than simply overwriting history.

SCREENSHOT 11

Change detection terminal output



Terminal output from demo_change.py showing CHANGE DETECTED and the old/new values.

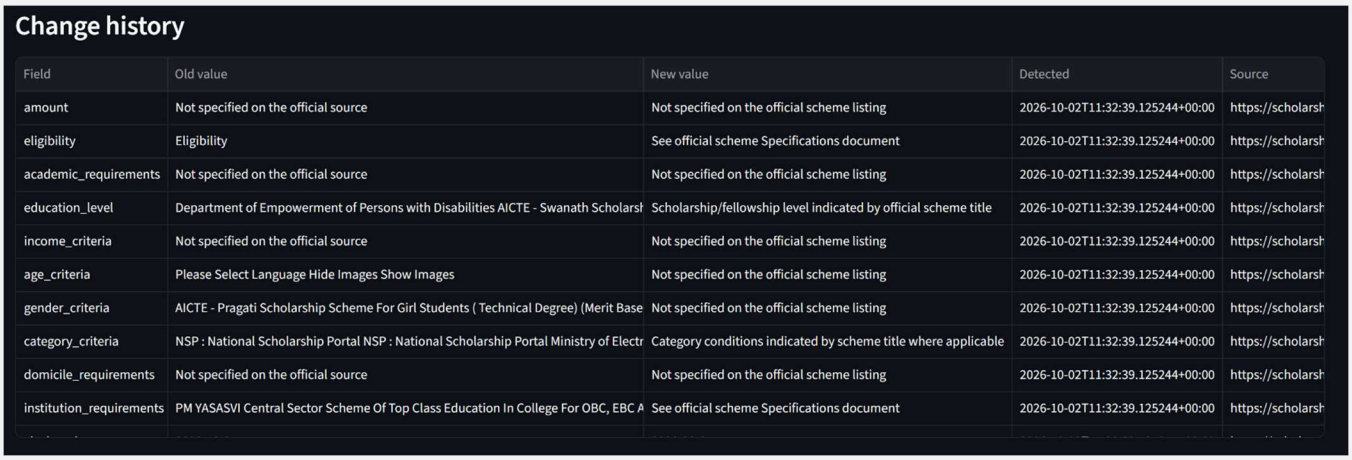
Change detection demonstration.

7.2 Refresh the dashboard

Return to Streamlit and refresh the page. Open the scholarship's change history.

SCREENSHOT 12

Change history



Scholarship details showing the recorded change event with old value, new value, date detected, source and evidence.

Persisted change history.

8. Demonstrate Stale / Expired Detection

The assignment asks the system to identify scholarships that are expired, no longer available, removed from the official source, unable to be re-verified, or contradicted by a newer official notification.

Statuses to explain

- ACTIVE – currently available according to the available evidence.
- EXPIRED – the application deadline has passed.
- NOT_YET_OPENED – a future opening date exists and the opportunity is not currently open.
- NO_LONGER_VERIFIABLE – the system cannot re-verify the scholarship from its primary source.
- REVIEW_REQUIRED – evidence is insufficient for the ≥95% verification threshold.

SCREENSHOT 13

Expired / stale records



Dashboard filtered to show expired or stale scholarships.

stale scholarship detection.

9. Run the Actual Live Crawler

After the local demo works, run the live crawler with an active internet connection. This is important because the assignment requires an actual working crawler, not only seeded/demo data.

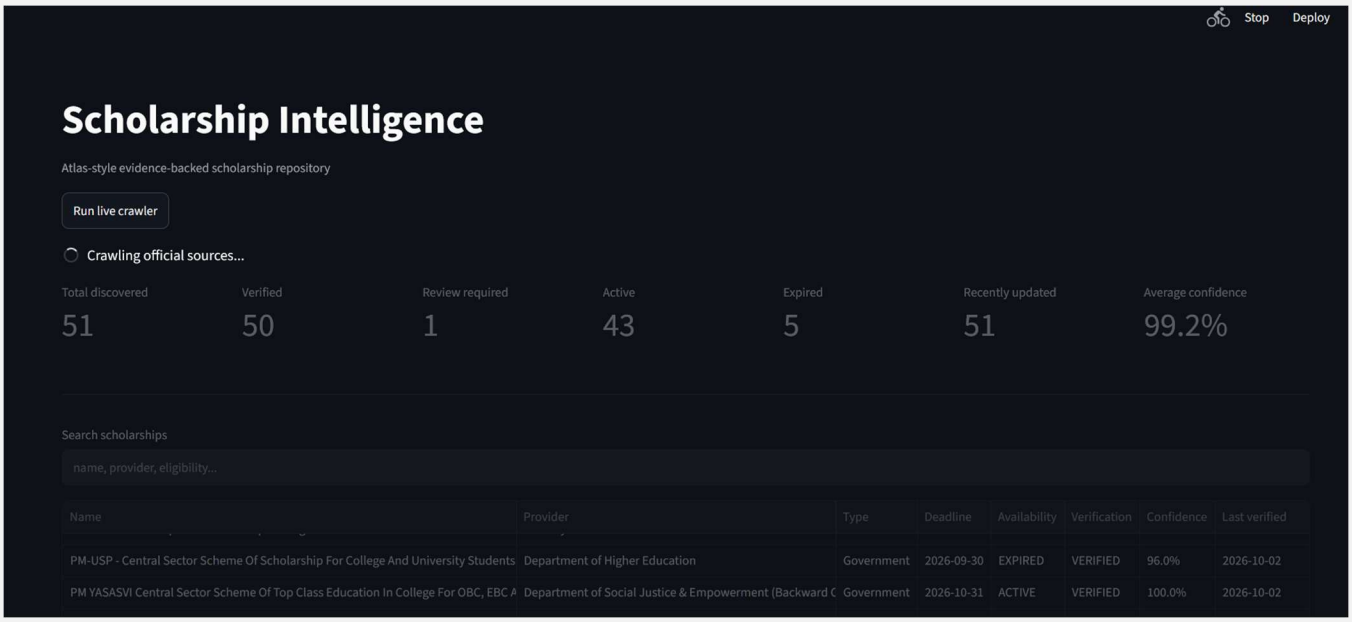
```
python -m app.cli crawl
```

The conceptual runtime flow is:

```
Seed sources
    ↓
Discovery
    ↓
Source classification / official-domain filtering
    ↓
Crawling
    ↓
Extraction
    ↓
Official-source verification
    ↓
Deterministic confidence scoring
    ↓
SQLite storage
    ↓
Change + stale detection
```

SCREENSHOT 14

Live crawler execution



Terminal while python -m app.cli crawl is running or after it completes, showing discovered/crawled/extracted records and any update messages.

Live crawler execution.

Important
If the live crawl fails because a website blocks automated requests, do not invent successful output. Record the actual result and use the included demonstration dataset for the product walkthrough while explaining the live-crawl limitation honestly.

10. Exact Demo Video Sequence

Use the following sequence in your short demonstration. Keep the terminal and browser visible enough for the reviewer to understand what is happening.

Step	Action	What to show / explain
1	Open terminal	Show the project folder and activated .venv.
2	Load demo data	Run python -m app.cli seed-demo.
3	Check assignment thresholds	Run python scripts/check_assignment.py and show PASS results.
4	Start dashboard	Run streamlit run app/dashboard.py.
5	Show dashboard counters	Show total discovered, verified, review required, active, expired, recently updated and average confidence.
6	Search	Search for a scholarship.
7	Open details	Show the complete structured record.
8	Show confidence	Show the confidence percentage and verification status.
9	Explain score	Open “Why this score?” or the scoring explanation.
10	Show evidence	Show official source and field-level evidence.
11	Show change history	Show previous changes for the scholarship.
12	Return to terminal	Prepare to demonstrate the update.
13	Run change demo	Run python scripts/demo_change.py.
14	Refresh dashboard	Show the resulting change history.
15	Run live crawler	Run python -m app.cli crawl.
16	Refresh dashboard	Show the resulting updated repository.

“This project implements the scholarship intelligence pipeline requested in the assignment. It starts from seed sources but performs discovery beyond a fixed list. It crawls relevant pages, extracts structured scholarship fields, identifies and verifies primary sources, and calculates a deterministic confidence score from evidence. Verified records require a score of at least 95%. The system stores field-level evidence so important values can be traced back to their source. On subsequent crawls, it compares new values with previous snapshots, records changes instead of silently overwriting history, and identifies stale or expired opportunities.”

11. Final Submission Checklist

- ☐ Project runs in Ubuntu/WSL.
- ☐ Virtual environment activates successfully.
- ☐ requirements.txt installs successfully.
- ☐ SQLite database initializes.
- ☐ Demo data loads.
- ☐ Assignment threshold checker passes.
- ☐ Tests pass.
- ☐ Streamlit dashboard opens.
- ☐ Dashboard metrics are visible.
- ☐ Scholarship search works.
- ☐ Scholarship details show structured fields.
- ☐ Confidence score is visible.
- ☐ Confidence explanation/evidence is visible.
- ☐ Official source URL is visible.
- ☐ Application URL is visible where available.
- ☐ Change history is visible.
- ☐ At least two change examples are demonstrated.
- ☐ At least two expired/stale examples are visible.
- ☐ Live crawler command is demonstrated.
- ☐ Second/repeated crawl is demonstrated.
- ☐ README is included.
- ☐ Technical note is included and within the requested maximum length.
- ☐ GitHub repository is public/accessibly shared as required.
- ☐ No paid APIs/services are used.
- ☐ No fabricated scholarship information is presented.

Screenshot checklist

Recommended screenshots to paste into this document:

1. Project folder in Ubuntu

2. Virtual environment activated

3. Assignment threshold checker

4. Pytest results

5. Dashboard home

6. Searchable scholarship list

7. Scholarship details

8. Confidence score

9. Why this score?

10. Source evidence

11. Change detection terminal

12. Change history

13. Expired/stale records

14. Live crawler execution

15. Second crawl/update

16. GitHub repository

14. Quick Command Reference

```
# Enter project
cd ~/atlas-scholarship-crawler

# Activate environment
source .venv/bin/activate

# Install dependencies
pip install -r requirements.txt

# Initialize DB
python -m app.cli init-db

# Load demo data
python -m app.cli seed-demo

# Check assignment requirements
python scripts/check_assignment.py

# Run tests
pytest -q

# Start dashboard
streamlit run app/dashboard.py

# Demonstrate change detection
python scripts/demo_change.py

# Run actual crawler
python -m app.cli crawl
```

12. Troubleshooting

Problem	Fix
python3: command not found	sudo apt update && sudo apt install -y python3 python3-pip python3-venv
No module named venv	sudo apt install -y python3-venv
pip install fails	Confirm .venv is active with: which python. It should point inside .venv.
streamlit: command not found	Activate .venv and run: pip install -r requirements.txt

Port 8501 already in use	Run: streamlit run app/dashboard.py --server.port 8502
Permission denied while activating	In Ubuntu, use: source .venv/bin/activate. Do not use the Windows PowerShell activation command.
Live crawl returns little data	Check internet access, source availability and site anti-bot behaviour. Do not fabricate records.